

Closed-Loop Reactive Control for On-the-Move Grasping

Progress writeup — toolchain validation and reference QP on Apple Silicon

Anurag Paresh Shetye | ARMS Lab, IIT Bombay (visiting) | collab. with University of Osaka

31 May 2026

1 Goal

The system extends the open-loop pipeline of Kiyokawa et al. (IEEE/SICE SII 2025) for on-the-move grasping by introducing a high-rate reactive controller that consumes the predicted grasp pose (x, y, θ) in world frame and continuously recomputes arm joint velocities against a fused live base pose estimate, closing the loop on velocity-tracking error during the approach. The platform is the TurtleBot 3 Waffle (differential drive) with an OpenManipulator-X (4-DOF arm, 380 mm reach), a strictly more arm-constrained setup than the Toyota HSR used in the original paper.

The architectural target is the figure shared on 20 May 2026; the controller follows the Haviland-Corke Holistic formulation (IEEE RA-L 2022), adapted to the constrained 4-DOF arm and integrated with a swappable grasp predictor (FCN reimplementation, GG-CNN, or OpenVLA-OFT). This document records the first concrete implementation step, taken on a development laptop ahead of moving to the lab Linux machine and physical hardware.

2 Setup

All work in this writeup runs on a MacBook Pro M3 Pro (18 GB RAM, Apple Silicon), with Python 3.9 in a project-local virtual environment. The intent of working on macOS first is to verify the controller-side toolchain in isolation, without any simulator or ROS runtime dependencies, so that subsequent integration on the lab Linux machine addresses one new concern at a time.

Repository.

- `github.com/orionop/mobile-grasping`
- Layout: `src/` (controller, predictor, pipeline modules), `Docs/` (this writeup, progress log), `tests/` (unit tests for the toolchain), `experiments/` (numbered experiment scripts), `notebooks/` (math derivations and sanity checks).
- Dependencies pinned in `pyproject.toml`; numpy held at <2 because the `roboticstoolbox-python` C extension is built against numpy 1.x.

Toolchain. The implementation builds on three open-source components:

- `roboticstoolbox-python` 1.1.1 (Corke, Haviland) — kinematic models, Jacobians, manipulator URDFs.
- `spatialmath-python` 1.1 — SE(3) algebra used for end-effector pose representation.

- `qp solvers` 4.8 with `osqp` 1.1 backend — the QP solver used to compute joint velocities each control step.

3 Code

The first experiment, `experiments/exp01_holistic_reference.py`, implements one control step of the Holistic QP on the reference Frankie platform (Franka Panda 7-DOF arm). The intent is purely to validate the solver pipeline end-to-end on Apple Silicon before adapting any of the formulation to the constrained 4-DOF arm.

Decision variable. Joint velocities for the mobile manipulator, augmented with an end-effector slack vector:

$$x = \begin{bmatrix} \dot{q}_{\text{base}} \\ \dot{q}_{\text{arm}} \\ \delta \end{bmatrix} \in \mathbb{R}^{2+7+6} = \mathbb{R}^{15}.$$

Quadratic program.

$$\min_x \frac{1}{2} x^\top Q x + C^\top x \quad \text{s.t.} \quad J x = \nu_e^*, \quad X^- \leq x \leq X^+,$$

where the augmented Jacobian is $J = [J_{\text{base}} \quad J_{\text{arm}}(q) \quad \mathbb{I}_{6 \times 6}]$ and the slack penalty in Q is set high ($1 \text{ e } 6$) so that the equality constraint is enforced tightly in this reference test; the full adaptive-weight scheme from RA-L 2022 is left for the next iteration. The arm joint velocities are bounded to $\pm 1.5 \text{ rad/s}$ and the base virtual joints to $\pm 0.3 \text{ m/s}$.

Test case. The Panda is placed in the toolbox-provided `qr` ready configuration. The desired end-effector twist in the world frame is $\nu_e^* = (0.1, 0, 0, 0, 0, 0) \text{ m/s}$, i.e. pure $+x$ translation with no rotation, no y/z motion. The QP is solved once and the realised twist is reconstructed as $\nu_e^{\text{realised}} = J_{\text{arm}}(q) \dot{q}_{\text{arm}}$ to check tracking quality.

4 Result

Quantity	Value
End-effector start position (Panda <code>qr</code>)	(0.484, 0.000, 0.413) m
Desired end-effector twist	(0.1, 0, 0, 0, 0, 0) m/s
Base joint velocities returned	(0, 0)
Arm joint velocities returned (rad/s)	(0, 0.308, 0, 0.297, 0, 0.012, 0)
Slack returned (6-D)	≈ 0
Realised end-effector twist	(0.1, 0, 0, 0, 0, 0) m/s
Tracking error (L2 norm)	2×10^{-6}

Table 1: One-step output of the reference Holistic QP on the Panda. The solver returns joint velocities that realise the requested end-effector twist to within 2×10^{-6} , with zero slack absorbed at the chosen penalty.

Three observations are worth recording:

- The QP returns a feasible, well-conditioned solution in well under 10ms on the M3 Pro, comfortably below the control period required for the ~ 200 Hz target rate.
- Only three of the seven arm joints (joints 2, 4, 6) carry motion. At the **qr** configuration with a pure $+x$ translational request, this is the kinematically minimal solution and indicates the Jacobian and the cost weighting are producing sensible joint allocations.
- Zero slack at the chosen $1\text{e}6$ penalty confirms that the equality constraint is being enforced as intended. When the penalty is lowered to $1\text{e}2$, the solver trades joint motion against tracking error in the expected direction (slack absorbs $\sim 1.6\%$ of the desired velocity), which validates the Holistic slack mechanism is wired correctly.

5 What this validates and what remains

The Day 1 result is narrow but iron-clad: the controller-side toolchain (**roboticstoolbox-python**, **qp solvers**, **osqp**) works correctly on Apple Silicon, the augmented Jacobian formulation from Haviland-Corke RA-L 2022 implements cleanly in Python, and the QP solver produces sensible outputs within a control-rate-compatible time budget.

Pending on the controller side. The reference experiment implements the QP structure only; the next iterations will add the remaining elements of the Holistic formulation:

- Manipulability gradient in the linear cost vector C , using the arm-only Yoshikawa measure $\hat{J}_m = [\mathbf{0}_b, J_m^a(q_a)]$.
- Adaptive cost weights λ_q and λ_δ as functions of the end-effector pose error magnitude $\|e\|$, so that the base contributes more when the goal is far and the arm contributes more when it is close.
- Velocity-damper inequality constraints for joint position limits, with the influence distance ρ_i and minimum distance ρ_s as parameters.
- Base orientation cost term θ_ϵ to keep the base facing the end-effector.
- Position-Based Servoing wrapper that converts a desired end-effector pose into the desired twist ν_e^* each step.

Pending on the platform side. The reference test uses the Panda model that ships with the toolbox. The constrained-platform adaptation will require a 4-DOF arm Jacobian for the OpenManipulator-X and a differential-drive virtual-joint representation for the TB3 Waffle base. With four arm degrees of freedom there is no redundancy slack against the full 6-D end-effector twist; the equality constraint will need to be projected onto the four task-space dimensions that Kiyokawa’s grasp pose actually specifies (x , y , z implicit, and θ around the vertical axis).

Pending on the system side. The simulator and ROS bring-up have not yet been started; that work moves to the lab Linux machine in the coming week. The intended sequence is: load the combined TB3 + OpenManipulator-X URDF in a simulator with the chosen ROS distribution; verify base-only and arm-only motion with stock controllers; integrate the QP controller as a ROS node that consumes a grasp pose and a live base pose estimate and publishes arm joint velocity commands; run a first open-loop versus closed-loop comparison on a single object at low base velocity (0.10 m/s) to establish a baseline before sweeping velocities.

6 Repository status

- `src/mobile_grasping/controller/qp_solver.py` — the Holistic QP wrapped as a reusable Python module (`HolisticQPSolver`, `HolisticQPConfig`).
- `experiments/exp01_holistic_reference.py` — the reference QP described above.
- `tests/test_imports.py` — toolchain sanity test.
- `Docs/writeup.tex`, `Docs/writeup.pdf` — this document.
- `Docs/progress_log.md` — internal running log of milestones.
- `pyproject.toml`, `.gitignore` — standard.

7 Update — ROS 1 Noetic workspace skeleton

After the Apple-Silicon toolchain validation above, the ROS package that will host the controller on the lab Linux machine has been authored on the development laptop and committed alongside the controller-side Python module. The lab PC currently runs ROS 1 Noetic with Gazebo Classic, which removes the gz-sim issues seen with ROS 2 Jazzy on the same machine and gives the most mature TurtleBot 3 plus OpenManipulator-X support.

7.1 Workspace layout

- `ros_ws/src/mobile_grasping_ros/` — a single Python-only catkin package with `package.xml`, `CMakeLists.txt`, and `setup.py`. Depends on `rospy`, the standard message packages, and on the ROBOTIS-GIT `turtlebot3_manipulation` stack at runtime.
- `scripts/controller_node.py` — subscribes to `/grasp_pose`, `/base_pose`, and `/joint_states`; publishes `/arm_velocity_cmd` at the configured rate (default 200 Hz). The node instantiates `HolisticQPSolver` from the package-local `qp_solver.py`. Velocity computation is currently a placeholder that publishes zeros; the equality-with-slack and bound-handling pieces are already in the solver, and the kinematics module that supplies $J_{\text{arm}}(q)$ and the Position-Based Servoing wrapper is the next addition.
- `scripts/predictor_node.py` — publishes a fixed `PoseStamped` on `/grasp_pose` at 30 Hz so the controller has a target during bring-up. This is the seam at which an FCN reimplementation, GG-CNN, or OpenVLA-OFT will later be plugged in without changing the controller node.
- `scripts/estimator_node.py` — subscribes to `/odom` and republishes the pose component as `/base_pose` at 50 Hz. The full fused estimator (wheel odometry, IMU, RGB-D visual odometry through an `robot_localization` EKF) replaces the body of this node later.
- `config/controller_params.yaml` — exposes the control rate, arm and base velocity bounds, slack penalty, and dimensions, so the same controller code can be retargeted between the Panda (reference) and OpenManipulator-X (deployment) by changing the YAML.
- `launch/pipeline.launch` — brings up only the three Python nodes against an existing `/odom` and `/joint_states` publisher.
- `launch/sim.launch` — composes with `turtlebot3_manipulation_gazebo`'s empty-world launch for a full simulation bring-up.
- `ros_ws/INSTALL.md` — explicit apt and source commands for the lab PC's first-time setup of TurtleBot 3, OpenManipulator-X, the `turtlebot3_manipulation` stack from ROBOTIS-GIT, and the Python solver dependencies (`qpsovers`, `osqp`). The lab machine is shared with other users, so the setup is deliberately isolated: project files live under `~/Desktop/anurag_ws/mobile-grasp`; the catkin workspace is the in-repo `ros_ws/` folder (no contact with any other workspace on the machine), and Python dependencies install at `--user` level. The only system-state change is the optional apt installs in step 2 of `INSTALL.md`, each of which is gated by a pre-flight check so only packages not already present are installed.

7.2 Status

Authored and pushed: package skeleton, three Python node stubs, launch and configuration files, install documentation. Not yet exercised on the lab machine: `catkin_make`, Gazebo bring-up, topic flow verification. The intent of the next session is to build the workspace, confirm that `roslaunch mobile_grasping_ros sim.launch` spawns the TB3 Waffle with the OpenManipulator-X in Gazebo Classic, and capture topic-rate output for the next update.